SID: 1155212799 Name: Chan Hei Lun

CSCI 2100 Data Structures

Assignment 2
Due Date: 3 November 2025

Written Exercises

1. Based on the header file list.h, write the following functions as recursive functions.

   a) Write the function count that returns the number of occurrences of the element in the second argument in the first argument list.
   int count(listADT, listElementT);

   Hint: Count the number of occurrences of the element in the tail.

```
#include "list.h"
int count(listADT list, listElementT element) {
if (ListIsEmpty(list)) {
return 0;
}
if (Head(list) == element) {
return 1 + count(Tail(list), element);
} else {
return count(Tail(list), element);
}
}
```

   b) Write the function index that returns the position of the element in the second argument in the first argument list. If the element is the head of the list, then the function returns 1; if it is the head of the tail of the list, then the function returns 2; and so on. If the element is not a member of the list, then the function returns −1.
   int index(listADT, listElementT);

   Hint: If the element is not the head, find its position in the tail.

```
#include "list.h"
int list_index(listADT list, listElementT element) {
if (ListIsEmpty(list)) {
return -1;
}
if (Head(list) == element) {
return 1;
}
int sub_index = list_index(Tail(list), element);
if (sub_index == -1) {
return -1;
}
return 1 + sub_index;
}
```

c) Write the function removeE that returns a list that is the same as the first argument list, but with the first occurrence of the element in the second argument removed. If the element is not a member of the list, then the function returns the original list in the first argument

listADT removeE(listADT, listElementT);

Hint: If the element is the head, the function can simply return the tail.

```
#include "list.h"
#include <stdio.h>
#include <stdlib.h>
listADT removeE(listADT list, listElementT element) {
if (ListIsEmpty(list)){
return list;}
if (Head(list) == element) {
return Tail(list);}
else{listADT newTail = removeE(Tail(list), element);
return Cons(Head(list), newTail);}}
```

2. Based on the header file list.h, write the functions count, index and removeE in Question 1 as non-recursive functions.

Count:

```
#include "list.h"
int count(listADT list, listElementT element) {
int counter = 0;
listADT current = list;
while (!ListIsEmpty(current)) {
if (Head(current) == element) {
counter++;
}
current = Tail(current);
}
return counter;
}
```

Index:

```
#include "list.h"
int list_index(listADT list, listElementT element) {
int position = 1;
listADT current = list;
while (!ListIsEmpty(current)) {
if (Head(current) == element) {
return position;
}
current = Tail(current);
position++;
}
return -1; // Not found
}
```

removeE:

```
#include "list.h"
listADT removeE(listADT list, listElementT element) {
listADT reversed_prefix = EmptyList();
listADT current = list;
```

```
while (!ListIsEmpty(current) && Head(current) != element) {
reversed_prefix = Cons(Head(current), reversed_prefix);
current = Tail(current);}
if (ListIsEmpty(current)) {
return list;}
listADT result_tail = Tail(current);
listADT result = result_tail;
while (!ListIsEmpty(reversed_prefix)) {
result = Cons(Head(reversed_prefix), result);
reversed_prefix = Tail(reversed_prefix);}
return result;}
```

3. Write the function splitList that takes the first argument list and splits it into two: a list of all the elements at the odd positions (the second argument), and a list of all the elements at the even positions (the third argument):

```
void splitList(listADT list, listADT* oddL, listADT* evenL);
```

Note that the position of the head is 1; the position of the head of the tail is 2; and so on.

a) Write this function as a recursive function.

```
#include "list.h"
void splitList(listADT list, listADT* oddL, listADT* evenL) {
if (ListIsEmpty(list)) {
*oddL = EmptyList();
*evenL = EmptyList();
return;}
if (ListIsEmpty(Tail(list))){
*oddL = Cons(Head(list), EmptyList());
*evenL = EmptyList();
return;}
listADT odd_tail, even_tail;
listElementT h1 = Head(list);
listElementT h2 = Head(Tail(list));
splitList(Tail(Tail(list)), &odd_tail, &even_tail);
*oddL = Cons(h1, odd_tail);
*evenL = Cons(h2, even_tail);}
```

b) Write this function as a nonrecursive function.

```
#include "list.h"
static listADT reverseList(listADT list) {
listADT reversed = EmptyList();
listADT current = list;
while (!ListIsEmpty(current)) {
reversed = Cons(Head(current), reversed);
current = Tail(current);}return reversed;}
void splitList(listADT list, listADT* oddL, listADT* evenL) {
listADT rev_odd = EmptyList();
listADT rev_even = EmptyList();
listADT current = list;
int is_odd_pos = 1;
while (!ListIsEmpty(current)){
```

```
if (is_odd_pos){
rev_odd = Cons(Head(current), rev_odd);}else {
rev_even = Cons(Head(current), rev_even);}
current = Tail(current);
is_odd_pos = !is_odd_pos;}
*oddL = reverseList(rev_odd);
*evenL = reverseList(rev_even);}
```

4. Using splitList, write the mergesort function that sorts the argument list into ascending order:

listADT mergesort(listADT);

```
  #include "list.h"
#include <stdio.h>
void splitList(listADT list, listADT* oddL, listADT* evenL);
int ListLength(listADT L1);
static listADT merge(listADT l1, listADT l2) {
  if (ListIsEmpty(l1)) return l2;
  if (ListIsEmpty(l2)) return l1;
  if (Head(l1) <= Head(l2)) {
     return Cons(Head(l1), merge(Tail(l1), l2));
  }else {
     return Cons(Head(l2), merge(l1, Tail(l2)));
  }}
listADT mergesort(listADT list){
  if (ListLength(list) <= 1) {
     return list;}
  listADT oddL, evenL;
  splitList(list, &oddL, &evenL);
  listADT sorted_odd = mergesort(oddL);
  listADT sorted_even = mergesort(evenL);
  return merge(sorted_odd, sorted_even);}
```

5. Use **<u>the selection sort algorithm</u>** to sort the following sequence of integers into ascending order:

   7, 1, 1, 8, 6, 5, 6, 9

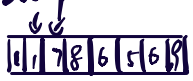
   Draw a series of diagrams to show clearly each step in the process of sorting.

First Pass

Swap 7 and 1 as min[7,1,1,8,6,5,6,9] = 1

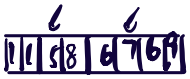
Second Pass

Swap 7 and 1 as min[7,1,8,6,5,6,9] = 1



Third Pass

Swap 7 and 5, as min[7,8,6,5,6,9] = 5

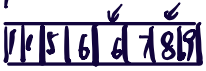


Forth Pass

Swap 8 and 6, as min[8,6,7,6,9] = 6

Fifth Pass

Swap 8 and 6 as min[8,7,6,9] = 6

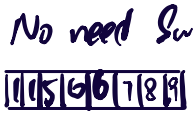


Sixth Pass

No need Swap anything as min[7,8,9] = 7

Seventh Pass

No need Swap anything as min[8,9] = 8



Final Sorted Array:

6. Write the following functions as recursive functions in C. You should use the list operations defined in list ADT shown in the lecture notes. You can simply call exit(EXIT_FAILURE) when error conditions are encountered. Note that your functions should work well with any correct implementation of list ADT.

   a) The function notMember that accepts an element as the first argument, and returns an int value 1 if the element is not a member of the list in the second argument, or 0 otherwise. For examples:
      - notMember(7, [8,1,5,3]) = 1
      - notMember(5, [7,6,5,4]) = 0
      - notMember(0, [0]) = 0
      - notMember(1, []) = 1

```
#include "list.h"
int notMember(int element, listADT list) {
if (ListIsEmpty(list)) {
return 1;
}
if (Head(list) == element) {
return 0;}
return notMember(element, Tail(list));}
```

   b) The function allDiff that accepts a listADT argument, and returns an int value 1 if all elements of the listADT arguments are different, or 0 otherwise. For examples:
      - AllDiff([]) = 1
      - AllDiff([2]) = 1
      - AllDiff([3,2,2,1]) = 0
      - AllDiff([3,2,3,1]) = 0
      - AllDiff([8,2,3,1]) = 1

   Hint: you may consider making use of the function notMember.

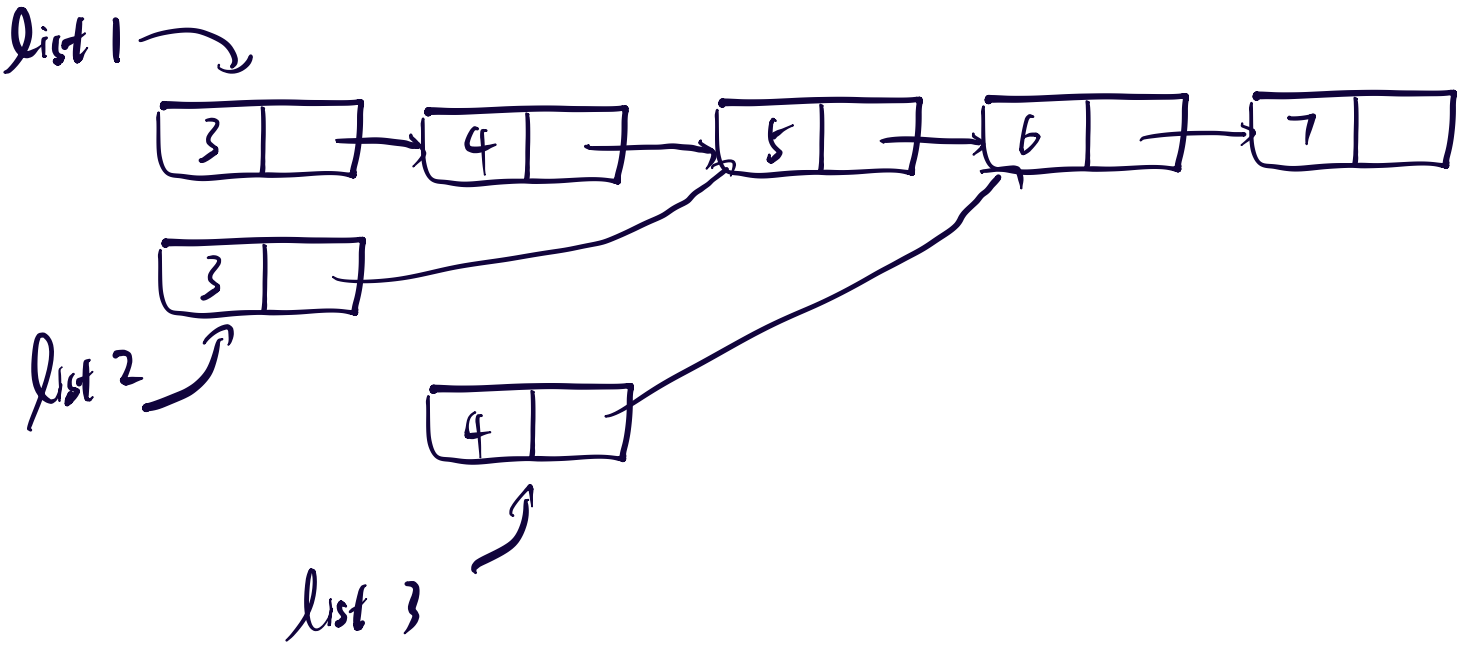
```
#include "list.h"
int notMember(int element, listADT list);
int allDiff(listADT list) {
if (ListIsEmpty(list) || ListIsEmpty(Tail(list))) {
return 1; }
if (notMember(Head(list), Tail(list)) == 0) {
return 0;}
return allDiff(Tail(list));}
```

   c) The function occurrence that accepts an int argument and a listADT argument, and returns the number of occurrences of the first argument in the list. For examples:
      - occurrence(4, [3,4,8,2,4]) = 2
      - occurrence(4, [3,5]) = 0
      - occurrence(1, []) = 0

```
#include "list.h"
int occurrence(int element, listADT list) {
if (ListIsEmpty(list)) {
return 0;}
if(Head(list)==element) {
return 1 + occurrence(element, Tail(list));}else{
return occurrence(element, Tail(list));}}
```

7. In the implementation version 2.0 of list ADT, we use a linked list-based implementation of lists. For the code segment shown below, draw a diagram to show the linked list-based implementation of lists **list1**, **list2** and **list3** after all the statements in the code segment are executed (you should assume that list1, list2 and list3 are all properly declared as listADT variables).

```
list1 = Cons(3, Cons(4, Cons(5, Cons(6, Cons(7, EmptyList())))));
list2 = Cons(Head(list1), Tail(Tail(list1)));
list3 = Cons(Head(Tail(list1)), Tail(Tail(list2)));
```



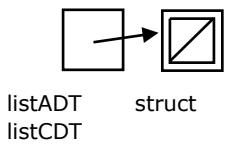
Programming Exercises (will conduct in other place)

8. There is a traditional, linked-list based implementation of the list ADT, which does not use structure sharing (as in version 2.0 in the lecture notes). Part of the implementation is shown below.

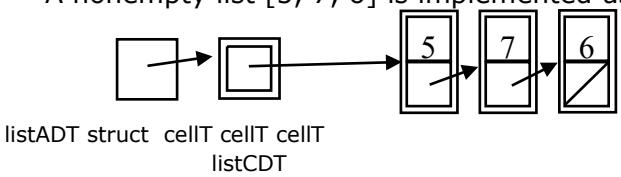
```
/*
* File: list.c
* Version 3.0 (traditional)
 */
#include "list.h"

typedef struct cellT { listElementT value; struct cellT *next; } cellT; struct
listCDT { cellT *listhead; };
```

An empty list [] is implemented as a NULL pointer stored in the structure member head:



A nonempty list [5, 7, 6] is implemented using a linked list of three elements:



Complete the implementation version 3.0.

9. There are two parts in this programming exercise.

   a) **(Part 1)** In Quiz 1 we have seen an implementation of the symbol table ADT. The keys are character strings (char*) and the values are pointed to by void pointers (void*). A mapping function forEachEntryDo is also provided, which applies the callback function in its first argument to each of the entries of the symbol table in its second argument.

```
/*
 * File: symtab.h
 */

typedef struct symtabCDT *symtabADT; typedef
void (*symtabFnT)(char*, void*);

symtabADT EmptySymbolTable(void);
void Enter(symtabADT, char*, void*);
void *Lookup(symtabADT, char*);
void Delete(symtabADT, char*); int
SymTabIsEmpty(symtabADT);
void forEachEntryDo(symtabFnT, symtabADT);
```

In this part of Assignment, you shall implement the symbol table ADT using Hash tables in file symtab.c. You should use open addressing hashing with double hashing as the collision resolution scheme. Part of the implementation file symtab.c is provided below.

```
/*
 * File: symtab.c
 */
#include <stdlib.h>
#include <string.h>
#include "symtab.h"
#define nrBuckets 200 // number of buckets

typedef struct cellT {char *key; void *value;} cellT; struct symtabCDT
{cellT *bucket[nrBuckets];};

symtabADT EmptySymbolTable() {    symtabADT table = (symtabADT)
malloc(sizeof(*table));    for (int i=0; i<nrBuckets; i++) table-
>bucket[i] = NULL;    return table;
}
```

You should use the following hash functions and complete the implementation in symtab.c.

```
int Hash(char *s) {    unsigned long hashcode = 0UL;    for (int i=0; s[i]!='\0';
i++) hashcode = (hashcode<<7) + (unsigned long) s[i];    return (int) (hashcode
% nrBuckets);
}

int Hash2(char *s) {    unsigned long hashcode = 0UL;    for (int i=0; s[i]!='\0';
i++) hashcode = (hashcode<<5) + (unsigned long) s[i];    return (int) (101 –
(hashcode % 101)); }
```

b) **(Part 2)** The Computer Programming Society is a newly founded Society in the Chan Tai Man Secondary School. To attract more students to join, the Society organizes a carnival for the whole school. In the carnival, there are ten games that students can play and win prizes. These ten games are:

- *Game 1: Pushing and Popping the Stacks*
- *Game 2: Joining and Leaving the Queues*
- *Game 3: Putting Symbols into Tables* • *Game 4: Where is the List Head?*
- *Game 5: Counting Leaves on Trees*
- *Game 6: Magic Sort*
- *Game 7: Search and Re-search* • *Game 8: Do You See the Graph?*
- *Game 9: Fighting the Python King*
- *Game 10: Oh Memory*

A student can choose to play some of these games and claim a prize. The type of prize a student can claim depends on what game(s) he/ she has played and the form he/ she is in. For example, a Form One student can claim a grand prize by playing only a few games, while a Form Six student must play all games to claim a grand prize.

Write a program that reads in names of students, the form they are in, the game they have played, and computes and prints the prize each student can claim.

The input to the program is a text file carnivalinput.txt. A sample input is shown below:

```
CHEUNG SIU YUN
4 5 7 2 1 6 7
CHAN LOK YIN
1 5 2 7 8 2 2
NG KA KA
1
LOK TAI YI
2 4 1 3 10
```

The information for a student is shown on two separate lines. The first line is the name of the student. The second line contains several integers. The first integer is the form (1 to 6) of the student, while the others indicate the games the student has played. Note that a student can play a game more than once. In the above example,

CHEUNG SIU YUN is a Form Four student and he has played the following games
- *Game 1: Pushing and Popping the Stacks*
- *Game 2: Joining and Leaving the Queues*
- *Game 5: Counting Leaves on Trees*
- *Game 6: Magic Sort*
- *Game 7: Search and Re-search* (twice)

CHAN LOK YIN is a Form One student and he has played the following games
- *Game 2: Joining and Leaving the Queues* (three times)
- *Game 5: Counting Leaves on Trees*
- *Game 7: Search and Re-search* • *Game 8: Do You See the Graph?*

NG KA KA is also a Form One student but he has not played any games.

Finally, LOK TAI YI is a Form Two student and he has played the following games

- *Game 1: Pushing and Popping the Stacks • Game 3: Putting Symbols into Tables • Game 4: Where is the List Head?*
- *Game 10: Oh Memory*

In your program, you should store the information in a symbol table. The keys in the symbol table are character strings, which are the names of the students (we assume that no two students have the same name). The "data" stored in the symbol table are void pointers pointing to structures of type **playerDataT** defined by:

typedef struct {int form; listADT gamesPlayed; prizeFnT p;} *playerDataT;

The first structure member **form** stores the form the student is in, and the second structure member **gamesPlayed** is a list of integers that stores the games the student has played. Finally, **p** is a function pointer pointing to a function that is used to determine the prize the student should get. The type **prizeFnT** is defined in the header file **prizeFnT.h** as

```
/* File: prizeFnT.h */

typedef enum {grandPrize, largePrize, mediumPrize, smallPrize, noPrize} prizeT;
typedef prizeT (*prizeFnT)(int, listADT);

prizeFnT prizeFunc123, prizeFunc45, prizeFunc6;
```

A **prizeFnT** function takes two arguments. The first is an integer that is the form the student is in, and the second is a list of integers that the student has played. There are three possible **prizeFnT** functions, all declared in **prizeFnT.h**:

- *prizeFunc123* should be stored in the structure member **p** if the student is a Form One, Form Two or Form Three student;
- *prizeFunc45* should be stored in the structure member **p** if the student is a Form Four or Form Five student;
- *prizeFunc6* should be stored in the structure member **p** if the student is a Form Six student.

In this way, in your program you can always call the function **p**, and either prizeFunc123, or prizeFunc45, or prizeFunc6 will be called to compute the prize the student should get. These three prizeFnT functions are implemented in the file **prizeFnT.c**. The outputs of these functions are shown below:

Function	Number of ***Different*** Games Played	Prize
prizeFunc123	1	Small Prize
	2	Medium Prize
	3	Large Prize
	4 or more	Grand Prize
prizeFunc45	1 or 2	Small Prize
	3 or 4	Medium Prize
	5 or 6	Large Prize
	7 or more	Grand Prize
prizeFunc6	1 to 3	Small Prize

	4 to 6	Medium Prize
	7 to 9	Large Prize
	10	Grand Prize

When the inputs are all read, and the table is ready, your program should print the names of students and the prize each student entitles. You can implement and make use of a mapping function **printAllNamesAndPrizes** that uses either prizeFunc123, prizeFunc45, or prizeFunc6 stored in the structure member **p** as a call back function. The following shows a sample output:

```
CHEUNG SIU YUN receives a Large Prize.
CHAN LOK YIN receives a Grand Prize.
NG KA KA receives no prize.
LOK TAI YI receives a Grand Prize.
```

— End of Assignment —